

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

192524319

I Harp VPS hnu N (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Harp VPS hnu N

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

SHORT ANSWER TEST

1. Construct language for the following Regular Expression:2

$$(a|b)^*a(a|b)|(a|b)$$

2. a) Illustrate the predictive parser for the following grammar. 4

$$S \rightarrow (L) | a$$

$$L \rightarrow L, S | S$$

3. Analyse, is it possible, by modifying the grammar in any way to construct predictive parser for the language of

$$S \rightarrow SS+ | SS* | a$$

String: "aa+a" 4

4. Eliminate left recursion from the grammar $S \rightarrow Sa|Sb|c$ 2
 5. The following grammar contains common prefixes, making it unsuitable for predictive parsing.

$$A \rightarrow abcD | abcE | f$$

Apply left factoring to remove the common prefixes and rewrite the grammar.5

6. Consider the following statement:10

$$Degree - F = Degree - C * 1.8 + 32$$

Show the representation of this statement after each phase of the compiler and explain.

7. Consider the following C code fragment:4

```
sum = 0;
```

```
for (i = 1; i < 5; i++)
```

```
    sum = sum + i;
```

Show the representation of the above code after each phase of the compiler.2

8. Consider the following grammar.

$$S \rightarrow aABb, A \rightarrow c|\epsilon, B \rightarrow d|\epsilon$$

Calculate FIRST and FOLLOW, Obtain the Predictive Parsing Table. not. Show the parser moves for the input string acdb.

9. Write a simple Lex program to recognize and count identifiers in a given input.2
 10. Write the rule to Eliminate Left Recursion and Left Factoring.2
 11. What are the actions performed by a shift-reduce parser? Explain each action with a suitable example.3
 12. Consider the following grammar:

$$S \rightarrow pBB \rightarrow qB|r$$

Find the leftmost derivation and construct the parse tree for the input string pqqr.3

13. Consider Grammar: $E \rightarrow E+T | T$ $T \rightarrow T * F | F$ $F \rightarrow (E) | id$

Eliminate left recursion and find FIRST and FOLLOW 5

14. For the following grammar, determine whether the string abcc is accepted using shift-reduce parsing. Show all shift and reduce operations.

$$S \rightarrow ABA \rightarrow aA \mid aB \rightarrow cB \mid c$$

15. For the following grammar, verify whether the string **abbc** is valid using shift-reduce parsing. 10

$$S \rightarrow aABA \rightarrow bA \mid bB \rightarrow c$$

Show the complete sequence of stack, input, and parser actions.

16. Consider the following grammar: 10

$$S \rightarrow SS \rightarrow aABA \rightarrow bA \mid cB \rightarrow d$$

- Construct the canonical collection of LR(0) items.
- Construct the SLR(1) parsing table.

17. Consider the following grammar.

$$S \rightarrow iEtSS \mid a, S' \rightarrow eS \mid \varepsilon, E \rightarrow b$$

Obtain the Predictive Parsing Table. 5

18. Do left factoring in the following grammar: 3

$$A \rightarrow aAB \mid aA \mid a$$

$$B \rightarrow bB \mid b$$

19. Construct the LL(1) Parsing Table for the following grammars. 7

$$S \rightarrow ABCA \rightarrow a \mid \varepsilon B \rightarrow r \mid \varepsilon C \rightarrow b \mid \varepsilon$$

20. Construct the LL(1) Parsing Table for the following grammars. 7

$$S \rightarrow 1S \mid qABCA \rightarrow a \mid b bDB \rightarrow a \mid \varepsilon C \rightarrow b \mid \varepsilon D \rightarrow c \mid \varepsilon$$

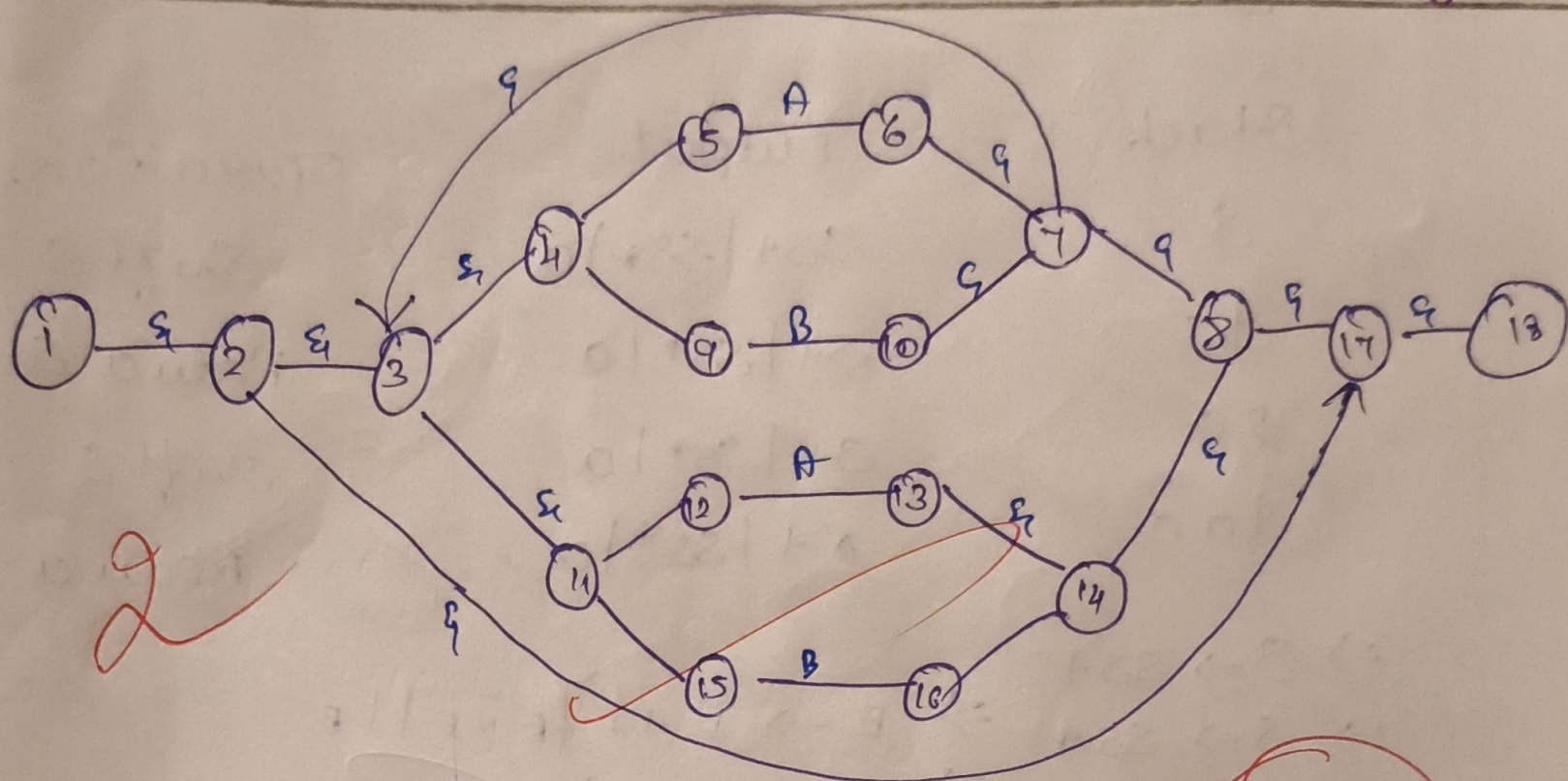
RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

CO2 AT2
Short answer test

HARI VISHNU N
192524319
CSA10102 - Compiler design.



2) $S \rightarrow (L)/a$
 $L \rightarrow L.S/S$

$S' \rightarrow a$

$L' \rightarrow LS$

$S' \rightarrow (L)/a$

$L \rightarrow SL'$

$L' \rightarrow S.L' (\epsilon)$

$\text{FIRST}(S) = \{ (\}$

$\text{FIRST}(L) = \{ \}$

$\text{FIRST}(L') = \{ \}$

$\text{Follow}(S) = \{ (, a, \$ \}$

$\text{Follow}(L) = \{ (, \$, a \}$

$\text{Follow}(L') = \{ (, \$, a \}$

4) $S \rightarrow sa | sb | a$

$S' \rightarrow saA'$

$S' \rightarrow saA' | sbA' | aA'$

$S' \rightarrow saA' | \epsilon$

$S' \rightarrow sbS'$

$S' \rightarrow bs' | \epsilon$

47
100

2

3) $S \rightarrow SS+ | SS* | a$
"aaa+a"

Stack

\$

\$s

\$a

\$a a

output

SS+|SS*|a

s+|ss*|a

s+|s*|a

s+|ss*|a

operation.

Shift s

Reduce $S \rightarrow a$

Shift s

Reduce $S \rightarrow a$

$\Rightarrow S \rightarrow SS+$

$\Rightarrow S \rightarrow SS*$

$\Rightarrow S \rightarrow a$

$\Rightarrow E \rightarrow E + E | E * E | E$

$\Rightarrow S' \rightarrow SaA'$

$\Rightarrow S' \rightarrow SaA' | SbA' | CA'$

$\Rightarrow S' \rightarrow SaA' | E$

$\Rightarrow S \rightarrow Sa | Sb | C //$

4.

~~Sum = 0;~~

~~for (i = 1, i < 5, i++)~~

~~Sum = Sum + i;~~

~~break;~~

$F = F'$

$F = C \times 1.8 + 32$

Syntax analysis

5) $A \rightarrow abcD | abcE | f$ apply left factoring.

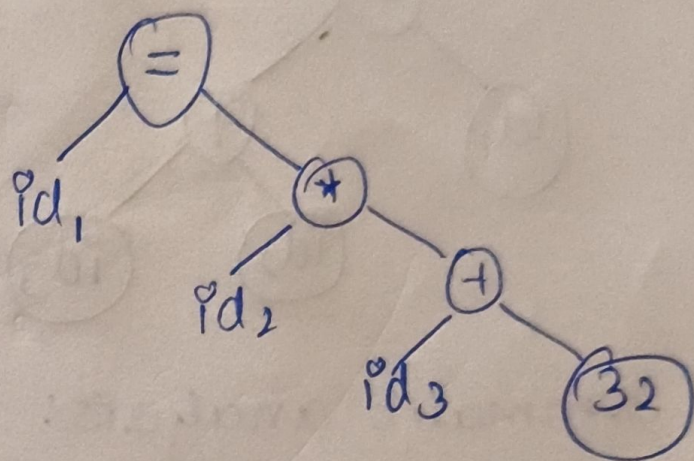
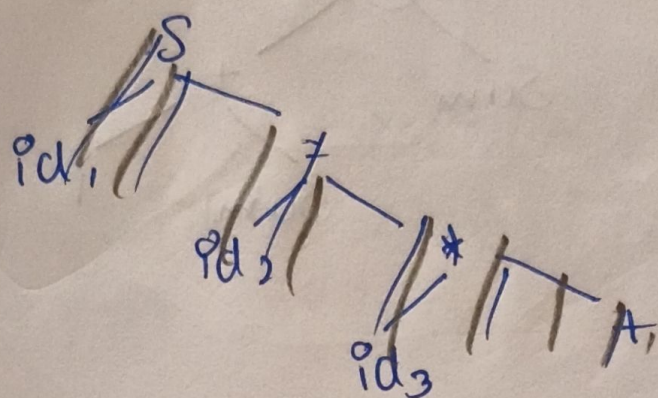
7

$$\text{Degree} - F = \text{Degree} \cdot C + 1.8 + 32$$

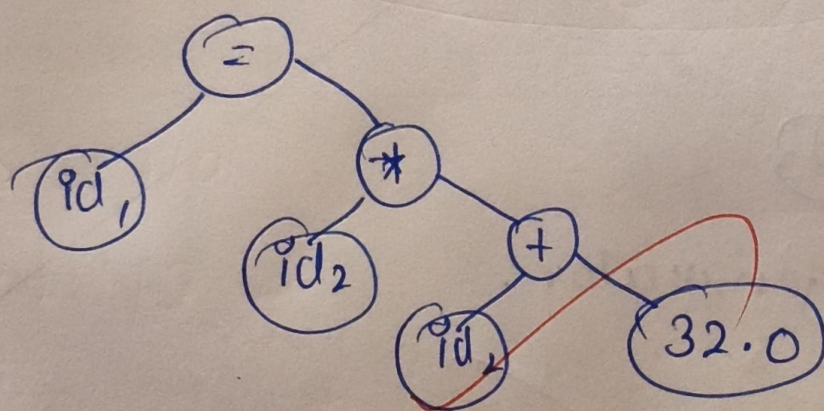
① Phases of compiler :-

$\langle \text{id}_1 \rangle \langle = \rangle \langle \text{id}_2 \rangle \langle * \rangle \langle \text{id}_3 \rangle \langle + \rangle \langle 32 \rangle$

② Syntax analysis:



③ Semantic analysis:



④ Intermediate code generator:

temp1 = id1

temp2 = id2 * id3

temp3 = temp2 + 32.0

id = temp3

⑤ Code optimization:

temp1 = id2 * id3

temp2 = temp1 + 32.0

id = temp2

⑥ Code Generator:

MOV(r2) = id1, id2, id3

MUL(r1, *) = id2, id3

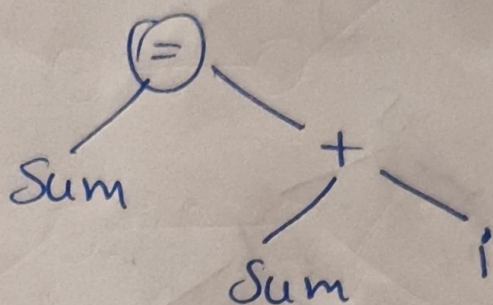
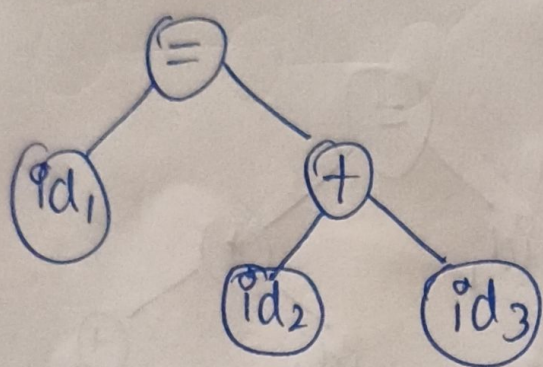
ADD(r2, +) = r1, 32.0

RESULT END

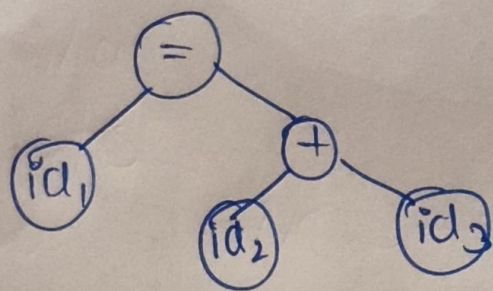
7) $S = \text{sum} + i$

① $\langle \text{sum} \rangle \langle = \rangle \langle \text{sum} \rangle \langle + \rangle \langle i \rangle$
 $\langle \text{id}_1 \rangle \langle = \rangle \langle \text{id}_2 \rangle \langle + \rangle \langle \text{id}_3 \rangle$

② Syntax Analysis:



③ Semantic analysis:



④ Intermediate code generator:

temp₁ = id₁

temp₂ = id₂ + id₃

temp₃ = temp₁

⑤ Code optimizer:

Temp₁ = id₂ + id₃

Temp₂ = Temp₁.

⑥ Code generator:

MOV(r₂) = id₂, id₃

ADD(r₁, +) = id₂ + id₃

FINISH.

$S \rightarrow aABb$

$A \rightarrow c/\epsilon$

$B \rightarrow d/\epsilon$

$FIRST(S) = \{c, \$\}$

$FIRST(A) = \{c, \epsilon\}$

$FIRST(B) = \{d, \epsilon\}$

$FOLLOW(S) = \{\$\}$

$FOLLOW(A) = \{\$, a\}$

$FOLLOW(B) = \{\$, b\}$

Predictive parsing table :-

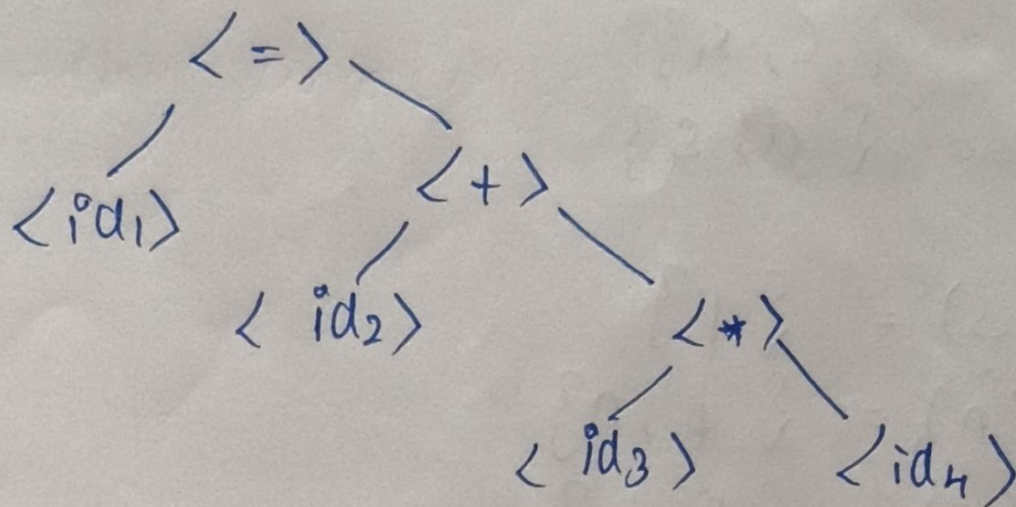
Terminals	a	b	c	d	ϵ	$\$$
S			$S \rightarrow aABb$			$S \rightarrow \epsilon$
A	$A \rightarrow \epsilon$		$A \rightarrow c/\epsilon$			$A \rightarrow \epsilon$
B		$B \rightarrow b/\epsilon$		$B \rightarrow d/\epsilon$		$B \rightarrow d/\epsilon$

- 1) write a sample lex pgm to recog and count identifiers.
- 2) write the rule to eliminate left recursion and left fact.
- 3) shift reduce parser - action how it will perform
- 4) consider grammar $E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid id.$ FIRST and FOLLOW.

9) Lex program:

Sample Input : $a = b + c * d$

$\langle pd_1 \rangle \langle = \rangle \langle pd_2 \rangle \langle + \rangle \langle id_3 \rangle \langle * \rangle \langle pd_4 \rangle$



Code:

$MOV(r_2) = pd_4, pd_3, pd_2, pd_1$

$ADD(r_2, \#1) \quad l=1$

$MUL(r_2, *) = pd_4 * pd_3$

$ADD(r_1, +) = r_2 + id_2$

$RES(r_1, \#=) = pd_1$

10) Left recursion:

$A \rightarrow A\alpha | B$

$A \rightarrow A\alpha$

$A \rightarrow B$

$B \rightarrow$ does not belong to A

eliminate left recursion by replacing their power of product with

$A \rightarrow BA'$

$A' \rightarrow \alpha A' | \epsilon$

Left factoring:

if $A \rightarrow \alpha A'$

$A' \rightarrow \beta / \gamma$

where α does not belong to A
 eliminate left factoring by overlapping their
 part

$$A \rightarrow \alpha A$$

Shift reduce Parser :-

operation :

⇒ Shift → It will move the symbols into the stack from output and input.

⇒ Reduce → Pop out RHS of production rule from stack and pushes LHS of production to stack.

⇒ Accept → It will include "\$" symbol, the I/P string is accepted.

3) ⇒ Error → the shift reduce is not possible display error.

* The priority of higher operator is higher than the operator at the stack that perform shift

* if the operator is less than or equal to then perform reduce.

3)

$$E \rightarrow E + T$$

$$E \rightarrow T$$

$$T \rightarrow T * F$$

$$T \rightarrow F$$

$$F \rightarrow (E) / id$$

$$E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id.$$

$$E' \rightarrow E + T \mid T$$

$$E \rightarrow E' T$$

$$E \rightarrow E' T \mid \Sigma$$

$$2) T \rightarrow T * F \mid F$$

$$T \rightarrow T' +$$

$$T \rightarrow * T' F \mid \Sigma$$

$$3) F \rightarrow (E) + id$$

$$F \rightarrow id \epsilon$$

$$F \rightarrow (E) + id).$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \epsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ +, \epsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$ \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$\text{FOLLOW}(T) = \{ \$,), + \}$$

$$\text{FOLLOW}(T') = \{ \$,), + \}$$

$$\text{FOLLOW}(F) = \{ \$, +,) \}$$

4

18) $A \rightarrow aAB|aA|a$ left factoring.
 $B \rightarrow bB|b$

$A \rightarrow aAB|aA|a$

$A \rightarrow aA'$

$A' \rightarrow aA'|\epsilon$

$B \rightarrow bB|b$

$B' \rightarrow bB'$

$B' \rightarrow bB'| \epsilon$

✓ 2

19. ~~Construct~~ Construct LL(1) Parsing Table for the following :-

$S \rightarrow ABC$

$A \rightarrow a|\epsilon$

$B \rightarrow r|\epsilon$

$C \rightarrow b|\epsilon$

X

20) LL(1)

$S \rightarrow IS\#|QABC$

$A \rightarrow a|bbD$

$B \rightarrow a|\epsilon$

$C \rightarrow b|\epsilon$

$D \rightarrow c|\epsilon$

X

19) $S \rightarrow ABC$

$A \rightarrow a|\epsilon$, $B \rightarrow r|\epsilon$, $C \rightarrow b|\epsilon$

$A' \rightarrow aA'|\epsilon$

$B' \rightarrow Br|\epsilon$

$C' \rightarrow bC'|\epsilon$

✓

$$\text{FIRST}(\delta) = \{A, B\}$$

$$\text{FIRST}(A) = \{a, \epsilon\}$$

$$\text{FIRST}(B) = \{r, \epsilon\}$$

$$\text{FIRST}(C) = \{b, \epsilon\}$$

$$\text{FOLLOW}(\delta) = \{\$, A, B\}$$

$$\text{FOLLOW}(A) = \{\$, A, B, \epsilon\}$$

$$\text{FOLLOW}(B) = \{\$, r, \epsilon, A, B\}$$

$$\text{FOLLOW}(C) = \{\$, r, \epsilon, A, B\}$$

$$\delta \rightarrow \text{IS\#I} \delta \text{ABC}$$

$$A \rightarrow a|b|bD$$

$$B \rightarrow a|c$$

$$C \rightarrow a|c$$

$$D \rightarrow c|c$$

$$A' \rightarrow bDA'$$

$$\text{FIRST}(A) = \{b, \}$$

$$\text{FIRST}(B) = \{D\}$$

$$\text{FIRST}(\delta) = \{\}$$

$$\text{FIRST}(C) = \{c, \}$$

$$\text{FOLLOW}(A) = \{\$, \}$$

$$\text{FOLLOW}(B) = \{\$, D\}$$

$$\text{FOLLOW}(\delta) = \{\$, D, c\}$$

$$\text{FOLLOW}(C) = \{\$, D, c, A\}$$